

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерных технологий

Лабораторная работа 5

«Интерполяция функции»

по дисциплине «Вычислительная математика»

Вариант 13

Выполнила:

Павличенко Софья Алексеевна, Р3215

Проверила:

Малышева Татьяна Алексеевна

Санкт-Петербург 2025г.

Оглавление

Цель	3
Часть 1: Вычислительная реализация задачи.....	3
Часть 2: Программная реализация задачи	5
Листинг программы	5
Результат работы программы.....	9
Почему интерполяция функции ex даёт большую ошибку?	11
Выводы.....	12

Цель

Решить задачу интерполяции, найти значения функции при заданных значениях аргумента, отличных от узловых точек.

Часть 1: Вычислительная реализация задачи

x	y	X_1	X_2
1,10	0,2234	1,168	1,463
1,25	1,2438		
1,40	2,2644		
1,55	3,2984		
1,70	4,3222		
1,85	5,3516		
2,00	6,3867		

i	x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$	$\Delta^5 y_i$	$\Delta^6 y_i$
0	1,10	0,2234	1,0204	0,0002	0,0132	-0,0368	0,0762	-0,1313
1	1,25	1,2438	1,0206	0,0134	-0,0236	0,0394	-0,0551	
2	1,40	2,2644	1,0340	-0,0102	0,0158	-0,0157		
3	1,55	3,2984	1,0238	0,0056	0,0001			
4	1,70	4,3222	1,0294	0,0057				
5	1,85	5,3516	1,0351					
6	2,00	6,3867						

Воспользуемся первой формулой Ньютона для интерполирования вперед, так как $X_1 = 1,168$ лежит в левой половине отрезка:

$$t = \frac{X_1 - x_0}{h} = \frac{1,168 - 1,10}{0,15} \approx 0,453$$
$$N_6(X_1) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0 + \frac{t(t-1)(t-2)}{3!}\Delta^3 y_0 + \frac{t(t-1)(t-2)(t-3)}{4!}\Delta^4 y_0 + \frac{t(t-1)(t-2)(t-3)(t-4)}{5!}\Delta^5 y_0 + \frac{t(t-1)(t-2)(t-3)(t-4)(t-5)}{6!}\Delta^6 y_0$$

$$\begin{aligned}
y(1,168) &\approx 0,2234 + 0,453 * 1,0204 + \frac{0,453(0,453 - 1)}{2!} * 0,0002 \\
&+ \frac{0,453(0,453 - 1)(0,453 - 2)}{3!} * 0,0132 \\
&+ \frac{0,453(0,453 - 1)(0,453 - 2)(0,453 - 3)}{4!} * (-0,0368) \\
&+ \frac{0,453(0,453 - 1)(0,453 - 2)(0,453 - 3)(0,453 - 4)}{5!} * 0,0762 \\
&+ \frac{0,453(0,453 - 1)(0,453 - 2)(0,453 - 3)(0,453 - 4)(0,453 - 5)}{6!} * (-0,1313) \\
y(1,168) &\approx 0,693
\end{aligned}$$

i	x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$	$\Delta^5 y_i$	$\Delta^6 y_i$
-3	1,10	0,2234	1,0204	0,0002	0,0132	-0,0368	0,0762	-0,1313
-2	1,25	1,2438	1,0206	0,0134	-0,0236	0,0394	-0,0551	
-1	1,40	2,2644	1,0340	-0,0102	0,0158	-0,0157		
0	1,55	3,2984	1,0238	0,0056	0,0001			
1	1,70	4,3222	1,0294	0,0057				
2	1,85	5,3516	1,0351					
3	2,00	6,3867						

Центральная точка $x_0 = 1,55$, $X_2 = 1,463 < 1,55$, то есть $x < x_0 \rightarrow$ используем вторую интерполяционную формулу Гаусса.

$$\begin{aligned}
t &= \frac{X_2 - x_0}{h} = \frac{1,463 - 1,55}{0,15} \approx -0,58 \\
P_6(X_2) &= y_0 + t\Delta y_{-1} + \frac{t(t+1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-2} + \frac{(t+2)(t+1)t(t-1)}{4!}\Delta^4 y_{-2} \\
&+ \frac{(t+2)(t+1)t(t-1)(t-2)}{5!}\Delta^5 y_{-3} \\
&+ \frac{(t+3)(t+2)(t+1)t(t-1)(t-2)}{6!}\Delta^6 y_{-3}
\end{aligned}$$

$$\begin{aligned}
y(1,463) &\approx 3,2984 - 0,58 * 1,0340 + \frac{(-0,58)(-0,58 + 1)}{2!} * (-0,0102) \\
&+ \frac{(-0,58 + 1)(-0,58)(-0,58 - 1)}{3!} * (-0,0236) \\
&+ \frac{(-0,58 + 2)(-0,58 + 1)(-0,58)(-0,58 - 1)}{4!} * 0,0394 \\
&+ \frac{(-0,58 + 2)(-0,58 + 1)(-0,58)(-0,58 - 1)(-0,58 - 2)}{5!} * 0,0762 \\
&+ \frac{(-0,58 + 3)(-0,58 + 2)(-0,58 + 1)(-0,58)(-0,58 - 1)(-0,58 - 2)}{6!} \\
&* (-0,1313)
\end{aligned}$$

$$y(1,463) \approx 2,699$$

Часть 2: Программная реализация задачи

Листинг программы

```
import math

import numpy as np
from matplotlib import pyplot as plt
import scipy.optimize

def lagrange_polynomial(xs, ys, x):
    result = 0.0

    for i in range(n):
        term = ys[i]
        for j in range(n):
            if i != j:
                term *= (x - xs[j]) / (xs[i] - xs[j])
        result += term

    return result

def newton_polynomial_with_divided_differences(xs, x):
    diff_table = ys.copy()
    for i in range(1, len(xs)):
        for j in range(len(xs) - 1, i - 1, -1):
            diff_table[j] = (diff_table[j] - diff_table[j - 1]) / (xs[j] - xs[j -
i])

    result = diff_table[-1]
    for i in range(n - 2, -1, -1):
        result = result * (x - xs[i]) + diff_table[i]

    return result

def make_table_of_finite_differences(ys):
    diff_table = [ys]

    for order in range(1, len(ys)):
        prev_row = diff_table[-1]
        new_row = [prev_row[i + 1] - prev_row[i] for i in range(len(prev_row) -
1)]
        diff_table.append(new_row)

    return diff_table

def newton_polynomial_with_finite_differences(xs, x, diff_table, h):
    n = len(xs)
    middle = (xs[0] + xs[-1]) / 2

    closest_index = 0
    min_dist = abs(x - xs[0])
    for i in range(1, n):
        dist = abs(x - xs[i])
```

```

        if dist < min_dist:
            min_dist = dist
            closest_index = i

    if x <= middle:
        t = (x - xs[closest_index]) / h
        result_closest = diff_table[0][closest_index]
        mult = 1.0

        for i in range(1, len(diff_table)):
            mult *= (t - i + 1)
            if 0 <= closest_index < len(diff_table[i]):
                result_closest += mult * diff_table[i][closest_index] /
math.factorial(i)

        t = (x - xs[0]) / h
        result = diff_table[0][0]
        mult = 1.0

        for i in range(1, len(diff_table)):
            mult *= (t - i + 1)
            result += mult * diff_table[i][0] / math.factorial(i)

    else:
        t = (x - xs[closest_index]) / h
        result_closest = diff_table[0][closest_index]
        mult = 1.0

        for i in range(1, len(diff_table)):
            mult *= (t + i - 1)
            if 0 <= closest_index - i < len(diff_table[i]):
                result_closest += mult * diff_table[i][closest_index - i] /
math.factorial(i)

        t = (x - xs[n - 1]) / h
        result = diff_table[0][n - 1]
        mult = 1.0

        for i in range(1, len(diff_table)):
            mult *= (t + i - 1)
            if 0 <= n - i - 1 < len(diff_table[i]):
                result += mult * diff_table[i][n - i - 1] / math.factorial(i)

    return result, result_closest

def plot_interpolation(xs, ys, x_interp, y_interp, method_name, x_plot, y_plot):
    plt.figure()
    plt.plot(x_plot, y_plot, color='lightgray', linewidth=2)
    plt.plot(xs, ys, 'o', label='Узлы интерполяции', color='blue')
    plt.plot(x_interp, y_interp, 'ro', label=f'Интерполяция в x={x_interp:.3f}')

    plt.title(f'{method_name}')
    plt.xlim(x_min, x_max)
    plt.xlabel("x")
    plt.ylabel("y")
    plt.legend()
    plt.tight_layout()
    plt.grid(True, linestyle='--', alpha=0.8)
    plt.show()

```

```

# --- Исходные данные ---

n = 0
x = 0
table = []
funcs = {
    '1': lambda x: np.sin(x),
    '2': lambda x: x ** 3,
    '3': lambda x: np.exp(x)
}

# --- Ввод данных ---

print('Вычислительная математика. Лабораторная работа 5: "Интерполяция функции".\n')

print("Как вы хотите ввести исходные данные?\n1 - с консоли\n2 - с файла\n3 - на основе функции")
v = input()
while v not in {'1', '2', '3'}:
    print('Выберите первый (1), второй (2) или третий (3) вариант')
    v = input()
print()

if v == '1':
    n = int(input("Сколько точек вы хотите ввести? "))
    while n < 0:
        n = int(input("Выберите значение больше 0: "))
    print("Введите построчно значения x и y через пробел:")
    for i in range(n):
        table.append(tuple(map(float, input().split())))
elif v == '2':
    print("Введите название файла:")
    with open(input().strip(), 'r') as file:
        lines = file.readlines()
        n = int(lines[0])
        for i in range(n):
            table.append(tuple(map(float, lines[i + 1].split()))))
else:
    print('1: sin(x)\n2: x^3\n3: e^x')
    func_number = input('Выберите функцию: ')
    while func_number not in {'1', '2', '3'}:
        func_number = input('Выберите первую (1), вторую (2) или третью (3) функцию: ')
    print("\nВведите исследуемый интервал:")
    x0 = float(input('x_0 = '))
    xn = float(input('x_n = '))
    n = int(input("Введите количество точек на интервале: "))
    xs = np.linspace(x0, xn, n)
    for x, y in zip(xs, funcs[func_number](xs)):
        table.append((x, y))
x = float(input('Введите точку интерполяции: '))
if v == '3':
    print(f"Реальное значение функции в точке интерполяции:",
    funcs[func_number](x))
table = sorted(table)
print()

```

```

# --- Интерполяция ---

xs = [xi for xi, _ in table]
ys = [yi for _, yi in table]
h = xs[1] - xs[0]

x_min_raw = min(min(xs), x)
x_max_raw = max(max(xs), x)
padding = 0.05 * (x_max_raw - x_min_raw)
x_min = x_min_raw - padding
x_max = x_max_raw + padding
x_plot = np.linspace(x_min, x_max, 1000)

lagrange_result = lagrange_polynomial(xs, ys, x)

plot_interpolation(xs, ys, x, lagrange_result, "Многочлен Лагранжа", x_plot,
[lagrange_polynomial(xs, ys, xi) for xi in x_plot])

print(f"\nРезультаты интерполяции в точке x = {x}:")
print(f"  1. Многочлен Лагранжа:                y ≈
{lagrange_result:.8f}")

if not all(abs(xs[i + 1] - xs[i] - h) < 1e-9 for i in range(len(xs) - 1)):
    newton_div_result = newton_polynomial_with_divided_differences(xs, x)
    plot_interpolation(xs, ys, x, newton_div_result, "Многочлен Ньютона с
разделенными разностями", x_plot, [newton_polynomial_with_divided_differences(xs,
xi) for xi in x_plot])
    print(f"  2. Многочлен Ньютона с разделенными разностями: y ≈
{newton_div_result:.8f}")
    print("\n Конечные разности используются только на равномерной сетке!\n")
else:
    print("\n Разделенные разности используются на неравномерной сетке!\n")
    diff_table = make_table_of_finite_differences(ys)
    print("  2. Таблица конечных разностей ( $\Delta^k y$ ):")
    for i, row in enumerate(diff_table):
        print(f"       $\Delta^{\{i\}}$  y: " + ", ".join(f"{val:.5f}" for val in row))

    newton_fin_result = newton_polynomial_with_finite_differences(xs, x,
diff_table, h)
    plot_interpolation(xs, ys, x, newton_fin_result[0], "Многочлен Ньютона с
конечными разностями", x_plot, [result[0] for result in
(newton_polynomial_with_finite_differences(xs, xi, diff_table, h) for xi in
x_plot)])
    plot_interpolation(xs, ys, x, newton_fin_result[1], "Многочлен Ньютона с
конеч. разн. (с выбором ближ. слева x)", x_plot, [result[1] for result in
(newton_polynomial_with_finite_differences(xs, xi, diff_table, h) for xi in
x_plot)])
    print(f"      Многочлен Ньютона с конечными разностями:    y ≈
{newton_fin_result[0]:.8f}")
    if abs(newton_fin_result[0] - newton_fin_result[1]) > 1e-9:
        print(f"      (с выбором ближайшего слева x):        y ≈
{newton_fin_result[1]:.8f}")

```


Результат работы программы

Вычислительная математика. Лабораторная работа 5: "Интерполяция функции".

Как вы хотите ввести исходные данные?

- 1 - с консоли
- 2 - с файла
- 3 - на основе функции

3

1: $\sin(x)$

2: x^3

3: e^x

Выберите функцию: 1

Введите исследуемый интервал:

$x_0 = 0$

$x_n = 5$

Введите количество точек на интервале: 10

Введите точку интерполяции: 1.168

Реальное значение функции в точке интерполяции: 0.9199684533871396

Результаты интерполяции в точке $x = 1.168$:

1. Многочлен Лагранжа: $y \approx 0.91996802$

Разделенные разности используются на неравномерной сетке!

2. Таблица конечных разностей ($\Delta^k y$):

$\Delta^0 y$: 0.00000, 0.52742, 0.89619, 0.99541, 0.79522, 0.35584, -0.19057, -0.67966, -0.96432, -0.95892

$\Delta^1 y$: 0.52742, 0.36878, 0.09922, -0.20019, -0.43938, -0.54641, -0.48909, -0.28466, 0.00539

$\Delta^2 y$: -0.15864, -0.26956, -0.29940, -0.23919, -0.10703, 0.05732, 0.20443, 0.29005

$\Delta^3 y$: -0.11092, -0.02984, 0.06021, 0.13216, 0.16435, 0.14711, 0.08562

$\Delta^4 y$: 0.08108, 0.09006, 0.07194, 0.03219, -0.01724, -0.06149

$\Delta^5 y$: 0.00898, -0.01811, -0.03975, -0.04943, -0.04425

$\Delta^6 y$: -0.02709, -0.02164, -0.00968, 0.00519

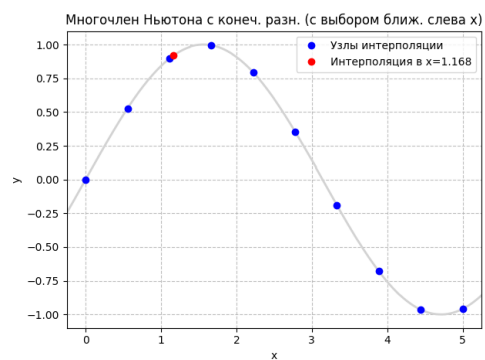
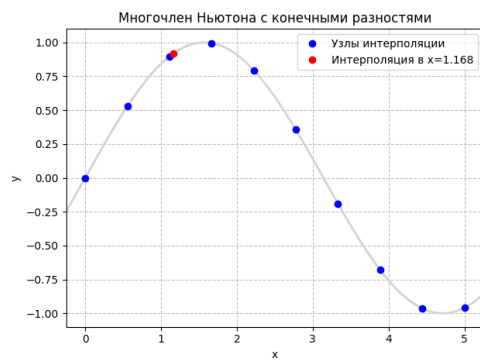
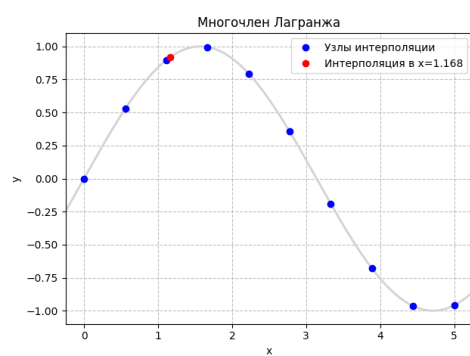
$\Delta^7 y$: 0.00545, 0.01196, 0.01487

$\Delta^8 y$: 0.00651, 0.00291

$\Delta^9 y$: -0.00360

Многочлен Ньютона с конечными разностями: $y \approx 0.91996802$

(с выбором ближайшего слева x): $y \approx 0.91999208$



Почему интерполяция функции e^x даёт большую ошибку?

Оценка интерполяционной погрешности в точке x имеет вид:

$$R_n(x) \leq \frac{M^{(n+1)}(x)}{(n+1)!} |(x-x_0)(x-x_1) \dots (x-x_n)|$$
$$M^{(n+1)}(x) = \max_{x \in [x_0; x_n]} f^{(n+1)}(x)$$

Все производные функции e^x равны самой функции:

$$f^{(n+1)}(x) = e^x$$

Вычислим погрешность для функции e^x :

$$M^{(11)}(0.5) = \max_{x \in [0; 10]} e^x = e^{10}$$
$$R_{10}(0.5) \leq \frac{e^{10}}{11!} |(0.5-0)(0.5-1)(0.5-2)(0.5-3)(0.5-4)(0.5-5)(0.5-6)(0.5-7)(0.5-8)(0.5-9)(0.5-10)| \approx 176.408$$

Ошибка интерполяции функции e^x в точке $x = 0.5$ оказывается очень большой, несмотря на использование 11 равномерно расположенных узлов на отрезке $[0, 10]$. Это объясняется следующими причинами:

Во-первых, функция e^x **быстро возрастает** на этом отрезке, а её производные всех порядков равны e^x . Поэтому значение производной 11-го порядка достигает максимума на правом конце интервала — $e^{10} \approx 22026$. В формуле оценки погрешности учитывается именно максимум производной на всём интервале, а не в окрестности точки $x = 0.5$, что сразу увеличивает потенциальную ошибку.

Во-вторых, хотя точка приближения находится в начале интервала, сам интерполяционный многочлен строится **глобально**, т.е. с учётом всех узлов. Даже в методе Ньютона с конечными разностями нельзя сказать, что мы используем только значение $y_0 = f(x_0)$. Уже начиная со второй разности, в расчётах участвуют значения функции в других точках, постепенно охватывая всё больше узлов — вплоть до $f(x_{10}) = e^{10}$. Таким образом, даже если точка приближения находится в начале интервала, построенный многочлен всё равно **зависит от поведения функции на всём отрезке**.

Кроме того, в формуле оценки ошибки входит произведение скобок $(x - x_i)$, которое в точке $x = 0.5$ содержит много крупных по модулю отрицательных множителей, что делает это произведение **очень большим по абсолютной величине**.

В результате, несмотря на наличие знака $\leq$ в формуле погрешности, реальное значение ошибки в точке $x = 0.5$ оказывается **близким к оценке**, так как все слагаемые (производная, произведение скобок) действительно велики.

Выводы

В ходе работы я изучила методы интерполяции функции. Вручную выполнила интерполяции с помощью формул Ньютона с конечными разностями и Гаусса. Программно на Python реализовала интерполяции с использованием формул Лагранжа и Ньютона с конечными и разделёнными разностями. Исследование показало, что точность интерполяции определяется распределением узлов, длиной интервала, расположением точки интерполяции и свойствами функции — в частности, поведением её производных и скоростью роста.